

Comparative Analysis of Seeding Strategies in K-means Clustering

Stability and Accuracy in High-Dimensional Manifolds

Gultaj Jafarova

April 28, 2026

Abstract

K-means is one of the most widely used clustering algorithms in machine learning. The performance of the algorithm is strongly influenced by the selection of the initial centroids. Different initialization methods can lead to different clustering results and affect the stability of the algorithm.

This report examines the effect of centroid initialization on the performance of K-means clustering. Two initialization methods, Random Seeding and K-means++, are compared to evaluate their impact on clustering quality and stability.

The K-means algorithm is implemented from scratch and tested on two datasets: Synthetic Blobs and MNIST. The Synthetic Blobs dataset is used to analyze clustering behavior on low-dimensional data, while the MNIST dataset provides a high-dimensional environment for evaluation.

Before clustering, data preprocessing is performed using StandardScaler to ensure that all features contribute equally to distance calculations. The performance of the two initialization methods is evaluated using inertia and standard deviation values obtained from multiple experimental runs.

The results show that both methods achieve similar performance on the synthetic dataset due to the overlapping cluster structure. However, K-means++ is generally expected to provide better results on more complex and high-dimensional datasets.

Overall, the study highlights the importance of centroid initialization and demonstrates how initialization strategies can influence the stability and effectiveness of K-means clustering.

Contents

1	Introduction	3
2	Theoretical Framework	3
2.1	The K-means Optimization Problem	3
2.2	Standard Seeding vs. K-means++	3
3	Methodology	4
3.1	Experimental Setup	4
3.2	Preprocessing	4
4	Results and Analysis	5
4.1	Stability Analysis	5
4.2	Synthetic Dataset Analysis	5
4.3	MNIST Dataset Analysis	6
5	Discussion	6
6	Conclusion	7
7	Appendix: Source Code	8
	References	11

1 Introduction

Clustering is an important technique in unsupervised machine learning. It is used to group similar data points into clusters without using labeled data. Clustering helps identify hidden patterns and structures within a dataset.

Among different clustering methods, K-means is one of the most popular algorithms because it is simple, efficient, and easy to implement. The algorithm works by dividing data into a predefined number of clusters and assigning each data point to the nearest centroid.

However, the performance of K-means depends heavily on the initial selection of centroids. Different starting points can lead to different clustering results.

This report focuses on the effect of centroid initialization in K-means clustering. Two initialization methods, Random Seeding and K-means++, are examined and compared. The objective is to analyze how these methods influence clustering performance, convergence, and stability. Understanding the effect of initialization is important because it can influence the final clustering outcome.

2 Theoretical Framework

2.1 The K-means Optimization Problem

The objective of K-means is to partition n observations into k clusters $S = S_1, S_2, \dots, S_k$ to minimize the within-cluster sum of squares (Inertia):

$$J = \sum_{i=1}^k \sum_{x \in S_i} |x - \mu_i|^2 \quad (1)$$

In this formula, J represents the inertia value, which is used to measure clustering quality. The term x denotes a data point, while μ_i represents the centroid of cluster i . The formula calculates the squared distance between each data point and its assigned centroid and then sums these distances across all clusters.

2.2 Standard Seeding vs. K-means++

Standard seeding selects k points randomly from the dataset. This method is simple, but the centroids may be placed too close to each other, which can reduce clustering quality. K-means++ improves initialization by selecting centroids more carefully. It spreads the centroids across the data space and often produces more stable clustering results.

3 Methodology

3.1 Experimental Setup

To ensure a robust evaluation, two distinct datasets were employed:

- **Synthetic Blobs:** This dataset contains 1,000 samples distributed across 8 cluster centers. A standard deviation of 2.5 was used to create overlapping clusters, making the clustering task more challenging. The dataset was selected because it provides a controlled environment for observing the behavior of different centroid initialization methods.

““

- **MNIST Dataset:** The MNIST dataset consists of handwritten digit images ranging from 0 to 9. In this study, 2,000 samples were used, with each image represented by 784 pixel features. This dataset was chosen because it is a well-known benchmark in machine learning and provides a high-dimensional environment for evaluating clustering performance. ““

3.2 Preprocessing

Data preprocessing is an important step before applying the K-means algorithm. Since K-means is based on distance calculations, all features should have a similar scale. If some features have much larger values than others, they can have a stronger influence on the clustering results.

For this reason, `StandardScaler` was used to normalize both the Synthetic Blobs and MNIST datasets. Standardization transforms the data so that each feature has a mean of 0 and a standard deviation of 1. As a result, the clustering process becomes more reliable and less sensitive to differences in feature scales.

$$z = \frac{x - \mu}{\sigma} \quad (2)$$

In this formula, x represents the original value, μ is the mean, and σ is the standard deviation. The transformed value is represented by z .

This step is especially important for the MNIST dataset because pixel values range from 0 to 255. Without normalization, features with larger values could dominate the distance calculations and negatively affect the clustering process.

4 Results and Analysis

4.1 Stability Analysis

4.2 Synthetic Dataset Analysis

The primary comparison metric is the distribution of inertia across 50 trials. Figure 1 shows the inertia distribution and variability obtained using Random Seeding and K-means++ on the synthetic dataset.

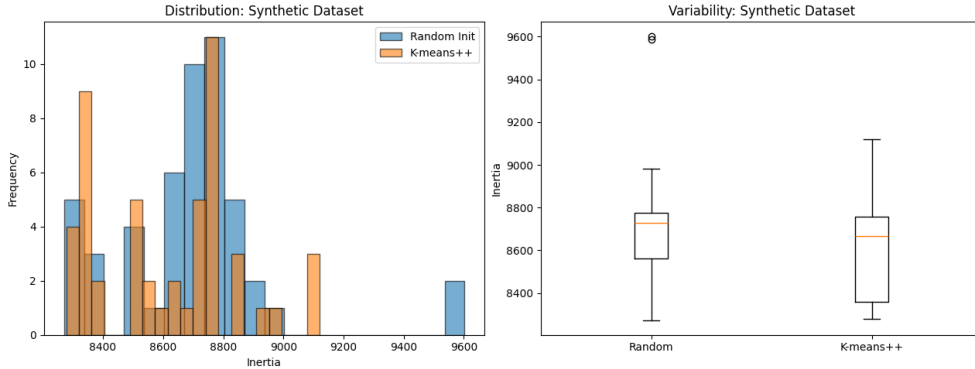


Figure 1: Inertia Distribution and Variability for Synthetic Data: Random vs. K-means++.

Initialization Method	Mean Inertia	Standard Deviation
Random Seeding	8631.71	228.84
K-means++	8679.65	279.37

Table 1: Comparative performance metrics over 50 trials on the Synthetic dataset.

The results show that both methods achieved similar performance on the synthetic dataset. The mean inertia values obtained from Random Seeding and K-means++ were very close, indicating that both initialization methods were able to produce comparable clustering results.

One reason for this is that the synthetic dataset contains overlapping clusters, which makes the clustering problem relatively simple. Even when the centroids are selected randomly, there is still a good chance that they will be placed near appropriate cluster centers.

As a result, the advantage of K-means++ is less noticeable in this particular dataset. Although K-means++ is designed to choose better initial centroids, both methods were able to find similar cluster structures and converge to comparable solutions in most experimental runs.

4.3 MNIST Dataset Analysis

The same experiment was also performed on the MNIST dataset. Since MNIST contains high-dimensional handwritten digit data, centroid initialization is expected to have a greater impact on clustering performance.

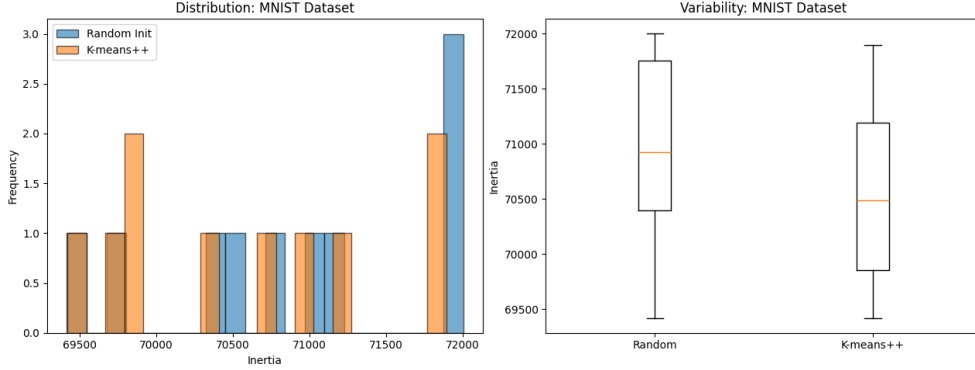


Figure 2: Inertia Distribution and Variability for MNIST Data: Random vs. K-means++.

Initialization Method	Mean Inertia	Standard Deviation
Random Seeding	70900.91	872.88
K-means++	70573.76	846.36

Table 2: Comparative performance metrics over multiple trials on the MNIST dataset.

The MNIST results show the behavior of K-means in a high-dimensional environment. The distributions and variability values indicate how centroid initialization can affect clustering performance. K-means++ generally provides more stable centroid placement and is often preferred for more complex datasets.

5 Discussion

The results obtained from the two initialization methods were similar. For high-dimensional datasets such as MNIST, Random Seeding may choose poor initial centroids, while K-means++ selects centroids more carefully and often produces better clustering results.

The numerical results also show that K-means++ achieved slightly lower inertia values on the MNIST dataset. This indicates that the algorithm was able to form more compact clusters.

However, the synthetic dataset used in this experiment contained overlapping clusters. Because of this, even randomly selected centroids were often placed near suitable cluster centers, reducing the difference between the two methods.

6 Conclusion

To conclude, this project examined the effect of centroid initialization on the performance of the K-means clustering algorithm. Two initialization methods, Random Seeding and K-means++, were compared in order to evaluate their impact on clustering quality and stability.

The experiments were performed on both a Synthetic dataset and the MNIST dataset. Multiple trials were conducted to obtain reliable and consistent results. The comparison was based on inertia values and the variability of the results across different runs.

The results showed that both methods achieved similar performance on the Synthetic dataset. This can be explained by the characteristics of the data. Since the dataset contained overlapping clusters and relatively low-dimensional features, random initialization was often able to find suitable centroid positions.

For the MNIST dataset, the results showed that K-means++ achieved slightly lower inertia values and more stable performance. This suggests that careful centroid selection becomes more important when working with high-dimensional datasets.

Although K-means++ did not provide a significant improvement on the Synthetic dataset, it remains an important initialization technique. By selecting centroids more carefully, it can reduce the risk of poor clustering results and improve convergence in more complex datasets.

This study also highlighted the importance of data preprocessing and proper centroid selection when applying clustering algorithms. Normalizing the data before clustering helped ensure that all features contributed equally to the distance calculations.

Another important observation is that the effectiveness of an initialization method often depends on the characteristics of the dataset. A method that provides only a small improvement on a simple dataset may offer a much greater benefit on larger and more challenging datasets.

Overall, the project provided a better understanding of how K-means clustering works, how initialization methods affect performance, and why K-means++ is often preferred for high-dimensional and challenging datasets. The implementation from scratch also helped strengthen practical knowledge of clustering algorithms and their behavior in different data environments.

7 Appendix: Source Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_blobs, fetch_openml
4 from sklearn.preprocessing import StandardScaler
5
6 class KMeansFromScratch:
7     def __init__(self, k=10, max_iters=100, init_method='k-
8         means++', tol=1e-4):
9         self.k = k
10        self.max_iters = max_iters
11        self.init_method = init_method
12        self.tol = tol
13        self.centroids = None
14        self.labels = None
15
16    def _initialize_centroids(self, X):
17        n_samples = X.shape[0]
18        if self.init_method == 'random':
19            indices = np.random.choice(n_samples, self.k,
20                replace=False)
21            return X[indices]
22        elif self.init_method == 'k-means++':
23            centroids = [X[np.random.choice(n_samples)]]
24            for _ in range(1, self.k):
25                dists = np.array([
26                    min([np.sum((x - c) ** 2) for c in
27                        centroids])
28                    for x in X
29                ])
30                probs = dists / np.sum(dists)
31                cumulative_probs = np.cumsum(probs)
32                r = np.random.rand()
33                for idx, p in enumerate(cumulative_probs):
34                    if r < p:
35                        centroids.append(X[idx])
36                        break
37            return np.array(centroids)
38
39    def fit(self, X):
```



```

37     self.centroids = self._initialize_centroids(X)
38     for _ in range(self.max_iters):
39         distances = np.sqrt(
40             ((X - self.centroids[:, np.newaxis])**2).sum(
41                 axis=2)
42         )
43         self.labels = np.argmin(distances, axis=0)
44         new_centroids = np.array([
45             X[self.labels == i].mean(axis=0)
46             if np.any(self.labels == i) else self.centroids
47             [i]
48             for i in range(self.k)
49         ])
50         # Convergence: check centroid displacement (Lloyd's
51         # criterion)
52         if np.allclose(self.centroids, new_centroids, atol=
53             self.tol):
54             break
55         self.centroids = new_centroids
56
57     def get_inertia(self, X):
58         inertia = 0
59         for i in range(self.k):
60             points = X[self.labels == i]
61             if len(points) > 0:
62                 inertia += np.sum((points - self.centroids[i])
63                     ** 2)
64         return inertia
65
66     def run_comparison(X, k, n_trials=30, title="Experiment"):
67         random_results, plus_results = [], []
68         for i in range(n_trials):
69             km_r = KMeansFromScratch(k=k, init_method='random')
70             km_r.fit(X)
71             random_results.append(km_r.get_inertia(X))
72
73             km_p = KMeansFromScratch(k=k, init_method='k-means++')
74             km_p.fit(X)
75             plus_results.append(km_p.get_inertia(X))

```

```

72
73 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
74 axes[0].hist(random_results, bins=20, alpha=0.6,
75               label='Random Init', edgecolor='black')
76 axes[0].hist(plus_results, bins=20, alpha=0.6,
77               label='K-means++', edgecolor='black')
78 axes[0].set_title(f'Distribution: {title}')
79 axes[0].set_xlabel('Inertia')
80 axes[0].set_ylabel('Frequency')
81 axes[0].legend()
82 axes[1].boxplot([random_results, plus_results],
83                 tick_labels=['Random', 'K-means++'])
84 axes[1].set_title(f'Variability: {title}')
85 axes[1].set_ylabel('Inertia')
86 plt.tight_layout()
87 plt.savefig('Figure_1.png', dpi=150)
88 plt.show()
89
90 print(f"Random->Mean: {np.mean(random_results):.2f}, "
91       f"Std: {np.std(random_results):.2f}")
92 print(f"K-means++->Mean: {np.mean(plus_results):.2f}, "
93       f"Std: {np.std(plus_results):.2f}")
94
95
96 if __name__ == "__main__":
97     X_syn, _ = make_blobs(n_samples=1000, centers=8,
98                           cluster_std=2.5, random_state=42)
99     run_comparison(X_syn, k=8, n_trials=50, title="Synthetic
100                   Dataset")
101
102     mnist = fetch_openml('mnist_784', version=1, as_frame=False
103                           )
104     X_mnist = StandardScaler().fit_transform(mnist.data[:2000])
105     run_comparison(X_mnist, k=10, n_trials=10, title="MNIST
106                   Dataset")

```

References

- [1] Arthur, D., & Vassilvitskii, S. (2007). K-means++: The Advantages of Careful Seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '07)*, pp. 1027–1035.
- [2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying.

Signed: _____

Date: April 28, 2026